TASK 11 – RECOMMEND AND DEFEND THE FINAL DESIGN

1. Aim

To recommend a suitable final design for the village milk payment calculator and justify the selection of data types, rounding rules, rate revision method and scalability.

2. Recommended Final Design

I recommend using Python functions, a separate rate-card module and the Decimal data type for the milk payment calculator.

The design should include:

- Decimal for accurate financial calculations.
- Functions for calculating the rate and farmer payment.
- Input validation to reject incorrect entries.
- Rounding each farmer's payment to two decimal places.
- A separate module to maintain the rate-card values.

This design is easy to understand, test, maintain and improve.

3. Data Types and Rounding Rule

Recommended data type: Decimal

Decimal is recommended instead of float because it provides accurate decimal arithmetic for money calculations.

For example:

- 0.1 + 0.2 using float may produce 0.30000000000000004.
- Decimal("0.1") + Decimal("0.2") produces exactly 0.3.

Therefore, Decimal is more suitable for calculating farmer payments.

Recommended rounding rule

Each farmer's payment should be rounded to two decimal places before adding it to the ten-day total.

This is recommended because each farmer receives an individual payment in rupees and paise. The total amount paid should equal the sum of the rounded individual payments.

4. Rate Revision Method

The rate card should be maintained separately from the main calculation program.

The current rate card contains:

Description	Value

Base rate	₹30.00
Standard fat	3.5%
Standard SNF	8.5%
Fat step value	₹0.40
SNF step value	₹0.30

If the co-operative changes the rate, the corresponding rate-card values can be updated in one place.

Using a separate module avoids changing the calculation logic in multiple programs.

5. Limitations of the Design

The design has some limitations:

1. It requires correct input values from the milk analyser or operator.
2. The program must be updated when the co-operative introduces new payment rules.
3. A simple local program does not automatically provide a central database or backup.
4. It does not include user login, access control or transaction security.
5. Large-scale use requires database storage and proper record management.

These limitations can be addressed by adding a database, backup facility and access control.

6. Suitability for District-Union Scale

The calculator can be extended to district-union scale because it uses reusable functions and a separate rate module.

The same calculation functions can process many farmer records.

For a larger system:

- Store farmer records in a database.
- Process records in batches.
- Use Decimal for financial calculations.
- Keep the rate card centrally managed.
- Maintain transaction records and backups.
- Test the system with large datasets.

The basic calculation logic remains the same, but database and system infrastructure are needed for large-scale operation.

7. Alternative Concept – Reading the Rate Card from a JSON File

Instead of writing the rate values directly inside the Python module, the values can be stored in a separate JSON file.

For example, a file named ratecard.json can contain:

```
{
   "base_rate": "30.00",
   "fat_step": "0.40",
   "snf_step": "0.30",
   "base_fat": "3.5",
   "base_snf": "8.5"
}
```

The Python program reads this file when it starts.

Advantages

- Rate values can be changed without editing the calculation functions.
- The same rate card can be used by different programs.
- Rate updates are easier to manage.

Disadvantages

- The program must read and validate the file at startup.
- A missing or incorrect file can cause errors.
- The file must be distributed and maintained correctly.

Complexity analysis

Method	Time	Space